

Proyecto: Generador de Folletos Corporativos con IA

Resumen

Desarrolla una aplicación que, dado el nombre de una empresa y su sitio web principal, **raspee** páginas relevantes (About, Careers, Customers, etc.), **seleccione enlaces útiles** y **genere un folleto** en Markdown (y PDF/HTML opcional) para clientes, inversionistas y talento potencial. Usa un LLM para:

1. clasificar enlaces relevantes y convertir URLs relativas en absolutas;
2. redactar el folleto a partir del contenido extraído.

Objetivos de aprendizaje

- Integrar **raspado web responsable** (requests + BeautifulSoup).
- Diseñar **prompts efectivos** y manejar **salidas estructuradas** (JSON).
- Encadenar pasos: scraping → filtrado de enlaces → compilación de contenidos → **brochure**.
- Empaquetar un proyecto reproducible (CLI o interfaz simple) con **logs**, **tests** y **documentación**.

Requisitos funcionales (MVP)

1. **Entrada:** nombre de la empresa y URL principal (p. ej., <https://empresa.com>).
2. **Scraping:** descargar la landing y extraer enlaces (ignorando <script>, <style>, imágenes, inputs).
3. **Selección de enlaces:** usar un modelo para decidir enlaces relevantes (About, Company, Careers/Jobs, Customers/Partners/Press, Blog si aporta “About” o cultura).
4. **Normalización:** convertir enlaces relativos a absolutos.
5. **Compilación:** descargar el HTML de cada enlace relevante, limpiar y consolidar texto.
6. **Brochure:** con un LLM, generar un folleto breve en **Markdown** (y opción de tono “formal” o “humorístico”).
7. **Salida:** guardar brochure.md y, opcionalmente, brochure.pdf/brochure.html.
8. **Modo demo:** si no hay clave de API, permitir un **modo “mock”** con respuestas simuladas para que todo el flujo corra.

Requisitos no funcionales

- **Ética del scraping:** respetar robots.txt cuando aplique, limitar la tasa de peticiones y definir headers.
- **Registro:** logs claros (INFO/ERROR) y manejo de errores (timeouts, 404).
- **Reproducibilidad:** requirements.txt o pyproject.toml, instrucciones de ejecución, y semilla para resultados deterministas cuando aplique.
- **Estructura:** código modular, funciones probadas y documentación.

Entregables

1. **Código fuente** con estructura ordenada.
2. **README.md** con:
 - Cómo ejecutar (con y sin API key).
 - Decisiones de diseño y límites conocidos.
 - Consideraciones éticas.
3. **Brochure** de 2–3 páginas en brochure.md (+ PDF/HTML).
4. **Demostración:** breve vídeo (2–4 min) o GIF mostrando el flujo.
5. **Casos de prueba** (mín. 3) en sitios distintos.
6. **Prompts** usados (archivos .md o comentarios en el código).

Criterios de evaluación (100 pts)

- **Funcionalidad end-to-end** (25): scraping → selección → compilación → brochure.
- **Calidad del folleto** (20): claridad, estructura, tono, cobertura (cultura, clientes, carreras si existen).
- **Diseño de prompts y manejo de JSON** (15): claridad, robustez, recuperación ante errores de formato.
- **Calidad de código** (15): modularidad, legibilidad, manejo de excepciones y logs.
- **Ética y cumplimiento** (10): respeto a robots.txt, rate limiting, disclaimers.
- **Reproducibilidad y docs** (10): README, dependencias, instrucciones.
- **Pruebas** (5): casos variados y evidencia de ejecución.

Hitos sugeridos

- **Día 1–2:** Setup, scraping básico + extracción de enlaces.
- **Día 3–4:** Clasificador de enlaces con LLM (JSON), normalización de URLs.

- **Día 5–6:** Compilación de contenidos y primer folleto en Markdown.
- **Día 7–8:** Pulido, modo mock, exportación a PDF/HTML, tests y demo.

Estructura recomendada del repositorio

```
brochure-ai/
├─ src/
|   ├─ scraping.py
|   ├─ link_selector.py
|   ├─ compiler.py
|   ├─ brochure.py
|   ├─ cli.py
|   └─ utils.py
├─ prompts/
|   ├─ link_system.md
|   ├─ link_user.md
|   └─ brochure_system.md
├─ tests/
|   ├─ test_scraping.py
|   └─ test_normalization.py
├─ data/          # (opcional) caché de HTML para debug
├─ outputs/       # brochure.md/pdf generados
├─ requirements.txt
├─ README.md
└─ .env.example
```

Guía técnica (plantilla mínima)

Dependencias

requests

beautifulsoup4

python-dotenv

openai # o el SDK que uséis en clase

markdown # si queréis convertir a HTML

pdfkit # o weasyprint, opcional para PDF

Variables de entorno

Archivo .env:

OPENAI_API_KEY=sk-...

MODEL=gpt-4o-mini

Ejemplo de flujo (pseudocódigo)

cli.py

uso: python -m src.cli --company "HuggingFace" --url "https://huggingface.co" --tone formal

```
def main(company, url, tone):
```

```
    html_main, links = scrape_and_extract(url)
```

```
    selected = select_relevant_links(url, links) # ← LLM JSON
```

```
    pages = { "landing": clean_text(html_main) }
```

```
    for item in selected["links"]:
```

```
        html = fetch(item["url"])
```

```
        pages[item["type"]] = clean_text(html)
```

```
    brochure_md = generate_brochure(company, pages, tone) # ← LLM
```

```
    save_markdown(brochure_md, f"outputs/{slug(company)}_brochure.md")
```

```
    export_pdf_optional(...)
```

Prompt para selección de enlaces (system)

Eres un asistente que recibe enlaces de un sitio y elige los más útiles para un folleto corporativo.

Responde SOLO en JSON con la forma:

```
{"links":[{"type":"about page","url":"https://..."}, {"type":"careers page","url":"https://..."}]}
```

Incluye solo enlaces de valor (About, Company, Careers, Customers, Partners, Press, Culture, Blog relevante).

Excluye: TOS, Privacy, email, login, carrito o cuentas.

Convierte cualquier enlace relativo a absoluto basado en el dominio de origen.

Prompt para el folleto (system)

Eres un asistente que analiza el contenido limpio de varias páginas de una empresa y redacta

un folleto breve en Markdown para clientes, inversores y candidatos.

Incluye: qué hace la empresa, propuesta de valor, productos/servicios, clientes o sectores, cultura y valores, carreras/beneficios si hay info, y un CTA. Usa subtítulos y viñetas.

Requisitos del folleto (estructura sugerida)

- **Título + slogan**
- **Qué hacemos** (1–2 párrafos)
- **Productos/Servicios** (viñetas)
- **Para quién** (industrias/beneficios)
- **Casos o clientes destacados** (si existen)
- **Cultura y valores** (viñetas/breve párrafo)
- **Carreras y beneficios** (si existe página Careers)
- **CTA** (cómo contactar, link)
- **Nota ética:** “Contenido generado a partir de fuentes públicas del sitio web en la fecha X; verificar antes de uso externo.”

Diferenciación por niveles

Básico (MVP)

- Flujo por línea de comandos.
- Un solo tono (formal).
- Sin PDF, solo Markdown.

Intermedio

- Selector de tono (formal/humorístico).
- Exportación a HTML y PDF.
- Caché local de páginas (data/) para evitar sobrecargar sitios.

Avanzado

- **UI** con Streamlit/Gradio.
- Detección y respeto de robots.txt + Retry-After.
- **Structured Outputs** (pydantic/JSON schema) para validar la respuesta del LLM.
- Soporte multilingüe (es/en).
- Tests de integración con responses/pytest para simular HTTP.

- Métricas: tiempo por etapa, nº de tokens, coste estimado.

Ética y aspectos legales (incluir en el README)

- Revisa robots.txt y términos del sitio; si no se permite el scraping de ciertas rutas, **no las consultes**.
- Limita la frecuencia de peticiones (p. ej., 1–2 req/seg máx.).
- Incluye un **User-Agent** identificable.
- No recolectes datos personales.
- Aclara que el folleto es un **resumen no oficial** y puede contener errores.

Pruebas sugeridas (mín. 3)

1. **Sitio corporativo estándar** (ej.: huggingface.co).
2. **Sitio con muchas rutas relativas**.
3. **Sitio minimalista** (pocos enlaces) para comprobar el modo “fallback” (si no hay Careers, omitir la sección).

Lista de verificación (antes de entregar)

- `python -m src.cli --company ... --url ...` genera `outputs/brochure.md`.
- Enlaces relativos → absolutos correctamente.
- JSON válido del selector (con manejo de errores si no lo es).
- README con instrucciones y ética.
- Modo mock sin API key funciona.
- 2–3 capturas de pantalla o un vídeo corto de la demo.

Extras (opcional para puntos extra)

- Detección de idioma del sitio y generación en ese idioma.
- Selector de “perfil de lector”: cliente vs inversor vs candidato (cambia secciones/prioridad).
- **Evaluación automática**: un script que calcule “cobertura” de secciones (About/Careers/Customers presentes).
- Integración con una API de conversión a PDF (WeasyPrint, wkhtmltopdf).

FAQ para alumnos

- **¿Qué pasa si el modelo no devuelve JSON limpio?**
Implementa reintentos y/o “JSON repair” (buscar llaves y parsear de nuevo).
- **¿Y si el sitio bloquea scraping?**
Respetar las reglas; usa otro sitio de prueba o el modo caché/mock.
- **¿Sin API key?**
Corre en modo mock: devuelve una salida JSON y un folleto de ejemplo para validar el pipeline.